

Autonomous Inference Offload

A zero-loss, local-first coding harness pairing local intelligence with frontier web models for high-throughput, private, and continuously audited codebases.

\$0.00

API INGRESS COST

100%

LOCAL DATA PRIVACY

 $O(N+M)$

TRIMMED LCS DIFFING

Zero-Loss

VERSION AUDIT RING

1. What is Offcoder in Plain English?

OVERVIEW

Modern software engineering with AI usually presents an agonizing tradeoff: either you send your entire intellectual property to expensive proprietary cloud APIs (paying per-token with unpredictable rate limits and latency), or you run local models (like *Qwythos*, *Qwen*, or *Ollama*) that operate privately on your LAN but occasionally miss subtle edge cases, architectural traps, or deep algorithmic security pitfalls.

Offcoder solves this completely. It gives your local model an apex coding harness equipped with live workspace read/write capabilities, real test and linter execution loops, durable architectural memory, and an automated **dual-engine web audit loop**. When your local model writes critical algorithms or UI layouts, it transparently offloads audits to frontier models (DeepSeek for algorithmic/security rigor; Kimi for visual design) through a real headless stealth browser session. Every draft, critique, and revision is immutably cached with instant visual diff tracking.

2. The Core Value Drivers

VALUE PROPOSITION

Zero Cloud Lock-In

SOVEREIGNTY

Runs entirely on your local machine and LAN. All codebase files, git AST indexes, and conversational logs remain anchored in your private SQLite memory ring and disk workspace.

Frontier Power at \$0

HEADLESS CDP

Harnesses frontier reasoning capabilities (DeepSeek R1 and Kimi) using automated, rate-resilient browser automation. Bypasses token metering while capturing elite code reviews.

Verified Closed Loop

EXECUTION FEEDBACK

No hallucinations in production. Offcoder immediately executes local linters, compilers, and test suites (pytest, npm test, swift build) feeding execution output directly back into context.

THE GOLDEN WORKFLOW FOR LOCAL DEVELOPERS

Explore Codebase → Draft Implementation → Offline Baseline Commit → Headless Frontier Audit → Trimmed Diff Resolution → Test Suite Verification. Offcoder ensures that your local AI has the exact same agency, rigor, and safety checks as an elite senior engineering team.

3. Multi-Process Tri-Domain Architecture

SYSTEM TOPOLOGY

Offcoder orchestrates three specialized, decoupled processes that communicate through high-throughput IPC and share an ACID SQLite write-ahead log (WAL) database:

RUNTIME DOMAIN	TECHNOLOGY STACK	NETWORK / IPC BOUNDARY	PRIMARY RESPONSIBILITIES
Native Cockpit Deck	SwiftUI / AppKit (Swift 5.9) macOS 13+	WebSocket Client :7171 HTTP Client :8080 / :8000	Real-time 60fps supervisor deck, trimmed LCS visual diff view, floating Web Reflection window, interactive chat, and project deliverables viewer.
Automation Daemon Core	Node.js 20+ / TypeScript Puppeteer + Tree-Sitter	WebSocket Server :7171 CDP Bridge :9222	Stealth browser automation (DeepSeek/Kimi/Gemini), screencast frame broker, git AST parsing, transactional task lease ring, and Prometheus metrics.
Dispatcher & Local Harness	Python 3.13 / FastAPI Asyncio + SQLite WAL	OpenAI Compatible :8000 LAN Model Gateway	OpenAI-compatible proxy, vision vs. text routing (DOM layout map extraction), background task verification worker, and exponential backoff retry manager.

4. The Zero-Loss Version & Diff Audit Ring

KEY ALGORITHMIC INNOVATION

- 01 Local Model Drafts Baseline (v0 Baseline)
When the local model generates a function or file, Offcoder automatically captures an immutable snapshot in the version registry. It records author, timestamp, and raw code before any external transmission.
- 02 Headless Offload & Web Reflection Window (consult_deepseek / consult_kimi)
The code is sent to DeepSeek or Kimi via a stealth browser session. Simultaneously, Offcoder's Web Reflection Viewer pops up on macOS, showing live, real-time prompts and streaming responses directly mirroring the headless browser process.
- 03 Trimmed Prefix/Suffix LCS Diff Generation (v1 Audited Revision)
Upon return, an $O(N+M)$ trimmed LCS algorithm matches unchanged prefix and suffix headers, isolating the exact mutation window. A unified line-by-line diff is created and displayed in the Cockpit with emerald additions and ruby deletions.
- 04 Closed-Loop Execution Verification (code_feedback)
The audited code is validated with native linters, compilers, and test suites. If tests fail, errors are fed back into context for automated repair.

Semantic Context Compression

QWYTHOS NATIVE

Context bloat is eliminated without losing traceability. When conversation depth grows, Qwythos semantically distills history into executive briefing anchors while literal verbose transcripts are permanently archived in `.agents/logs/conversations/`.

Project-Centric Artifacts Column

ISOLATED REPOS

Every codebase effort begins with a clean workspace inside `/Users/stevenjackson/code/qwythos-agent/projects/`. The Artifacts column shows clean dropdown files and deliverables specific to the active project, staying completely uncluttered.

ARCHITECTURAL SUMMARY

Offcoder represents the ultimate fusion of local data privacy, zero API costs, and frontier reasoning intelligence. By chaining immutable version diffs, headless audit offload, execution feedback, and native macOS supervision, developers gain superpowers without giving away their source code.